Libraries
=========

Set up your development environment to use the OpenAI API with an SDK in your preferred language.

This page covers setting up your local development environment to use the [OpenAI API](https://platform.openai.com/docs/api-reference). You can use one of our officially supported SDKs, a community library, or your own preferred HTTP client.

Create and export an API key

Before you begin, [create an API key in the dashboard](https://platform.openai.com/api-keys), which you'll use to securely [access the API](https://platform.openai.com/docs/api-reference/authentication). Store the key in a safe location, like a [`.zshrc` file](https://www.freecodecamp.org/news/how-do-zsh-configuration-files-work/) or another text file on your computer. Once you've generated an API key, export it as an [environment variable](https://en.wikipedia.org/wiki/Environment_variable) in your terminal.

macOS / Linux

Export an environment variable on macOS or Linux systems

```
export OPENAI_API_KEY="your_api_key_here"
```

Windows

Export an environment variable in PowerShell

```
setx OPENAI_API_KEY "your_api_key_here"
```

OpenAI SDKs are configured to automatically read your API key from the system environment.

Install an official SDK

JavaScript

To use the OpenAI API in server-side JavaScript environments like Node.js, Deno, or Bun, you can use the official [OpenAI SDK for TypeScript and JavaScript](https://github.com/openai/openai-node). Get started by installing the SDK using [npm](https://www.npmjs.com/) or your preferred package manager:

Install the OpenAI SDK with npm

```
npm install openai
```

With the OpenAI SDK installed, create a file called `example.mjs` and copy the example code into it:

Test a basic API request

```
import OpenAI from "openai";
const client = new OpenAI();

const response = await client.responses.create({
    model: "gpt-5.2",
    input: "Write a one-sentence bedtime story about a unicorn."
});

console.log(response.output_text);
```

Execute the code with `node example.mjs` (or the equivalent command for Deno or Bun). In a few moments, you should see the output of your API request.

[

Learn more on GitHub

Discover more SDK capabilities and options on the library's GitHub README.

](https://github.com/openai/openai-node)

Python

To use the OpenAI API in Python, you can use the official [OpenAI SDK for Python](https://github.com/openai/openai-python). Get started by installing the SDK using [pip](https://pypi.org/project/pip/):

Install the OpenAI SDK with pip

```
pip install openai
```

With the OpenAI SDK installed, create a file called `example.py` and copy the example code into it:

Test a basic API request

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5.2",
    input="Write a one-sentence bedtime story about a unicorn."
)

print(response.output_text)
```

Execute the code with `python example.py`. In a few moments, you should see the output of your API request.

[

Learn more on GitHub

Discover more SDK capabilities and options on the library's GitHub README.

](https://github.com/openai/openai-python)

.NET

In collaboration with Microsoft, OpenAI provides an officially supported API client for C#. You can install it with the .NET CLI from [NuGet](https://www.nuget.org/).

```
dotnet add package OpenAI
```

A simple API request to the [Responses API](https://platform.openai.com/docs/api-reference/responses) would look like this:

Test a basic API request

```
using System;
using System.Threading.Tasks;
using OpenAI;

class Program
{
    static async Task Main()
    {
        var client = new OpenAIClient(
            Environment.GetEnvironmentVariable("
OPENAI_API_KEY")
        );

        var response = await client.Responses.CreateAsync(new
ResponseCreateRequest
        {
            Model = "gpt-5.2",
            Input = "Say 'this is a test.'"
        });

        Console.WriteLine($"[ASSISTANT]:
{response.OutputText()}");
    }
}
```

To learn more about using the OpenAI API in .NET, check out the GitHub repo linked below!

[

Learn more on GitHub

Discover more SDK capabilities and options on the library's GitHub README.

](https://github.com/openai/openai-dotnet)

Java

OpenAI provides an API helper for the Java programming language, currently in beta. You can include the Maven dependency using the following configuration:

```
<dependency>
  <groupId>com.openai</groupId>
  <artifactId>openai-java</artifactId>
  <version>4.0.0</version>
</dependency>
```

A simple API request to [Responses API](https://platform.openai.com/docs/api-reference/responses) would look like this:

Test a basic API request

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;
import com.openai.models.responses.Response;
import com.openai.models.responses.ResponseCreateParams;

public class Main {
    public static void main(String[] args) {
        OpenAIClient client = OpenAIOkHttpClient.fromEnv();

        ResponseCreateParams params =
ResponseCreateParams.builder()
                .input("Say this is a test")
                .model("gpt-5.2")
                .build();

        Response response = client.responses().create(params);
        System.out.println(response.outputText());
    }
}
```

To learn more about using the OpenAI API in Java, check out the GitHub repo linked below!

[

Learn more on GitHub

Discover more SDK capabilities and options on the library's GitHub README.

](https://github.com/openai/openai-java)

Go

OpenAI provides an API helper for the Go programming language, currently in beta. You can import the library using the code below:

```
import (
  "github.com/openai/openai-go" // imported as openai
)
```

A simple API request to the [Responses API](https://platform.openai.com/docs/api-reference/responses) would look like this:

Test a basic API request

```
package main

import (
 "context"
 "fmt"

 "github.com/openai/openai-go/v3"
 "github.com/openai/openai-go/v3/option"
 "github.com/openai/openai-go/v3/responses"
)

func main() {
 client := openai.NewClient(
  option.WithAPIKey("My API Key"), // or set OPENAI_API_KEY in
your env
 )
```

```
 resp, err := client.Responses.New(context.TODO(),
openai.ResponseNewParams{
  Model: "gpt-5.2",
  Input: responses.ResponseNewParamsInputUnion{OfString:
openai.String("Say this is a test")},
 })
 if err != nil {
  panic(err.Error())
 }

 fmt.Println(resp.OutputText())
}
```

To learn more about using the OpenAI API in Go, check out the GitHub repo linked below!

[

Learn more on GitHub

Discover more SDK capabilities and options on the library's GitHub README.

](https://github.com/openai/openai-go)

Azure OpenAI libraries

Microsoft's Azure team maintains libraries that are compatible with both the OpenAI API and Azure OpenAI services. Read the library documentation below to learn how you can use them with the OpenAI API.

* [Azure OpenAI client library for .NET](https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/openai/Azure.AI.OpenAI)
* [Azure OpenAI client library for JavaScript](https://github.com/Azure/azure-sdk-for-js/tree/main/sdk/openai/openai)
* [Azure OpenAI client library for Java](https://github.com/Azure/azure-sdk-for-java/tree/main/sdk/openai/azure-ai-openai)
* [Azure OpenAI client library for Go](https://github.com/Azure/azure-sdk-for-go/tree/main/sdk/ai/azopenai)

* * *

Community libraries

The libraries below are built and maintained by the broader developer community. You can also [watch our OpenAPI specification](https://github.com/openai/openai-openapi) repository on GitHub to get timely updates on when we make changes to our API.

Please note that OpenAI does not verify the correctness or security of these projects. **Use them at your own risk!**

C# / .NET

* [Betalgo.OpenAI](https://github.com/betalgo/openai) by [Betalgo](https://github.com/betalgo)
* [OpenAI-API-dotnet](https://github.com/OkGoDoIt/OpenAI-API-dotnet) by [OkGoDoIt](https://github.com/OkGoDoIt)
* [OpenAI-DotNet](https://github.com/RageAgainstThePixel/OpenAI-DotNet) by [RageAgainstThePixel](https://github.com/RageAgainstThePixel)

C++

* [liboai](https://github.com/D7EAD/liboai) by [D7EAD](https://github.com/D7EAD)

Clojure

* [openai-clojure](https://github.com/wkok/openai-clojure) by [wkok](https://github.com/wkok)

Crystal

* [openai-crystal](https://github.com/sferik/openai-crystal) by [sferik](https://github.com/sferik)

Dart/Flutter

* [openai](https://github.com/anasfik/openai) by [anasfik](https://github.com/anasfik)

Delphi

* [DelphiOpenAI](https://github.com/HemulGM/DelphiOpenAI) by [HemulGM](https://github.com/HemulGM)

Elixir

* [openai.ex](https://github.com/mgallo/openai.ex) by [mgallo](https://github.com/mgallo)

Go

* [go-gpt3](https://github.com/sashabaranov/go-gpt3) by [sashabaranov](https://github.com/sashabaranov)

Java

* [simple-openai](https://github.com/sashirestela/simple-openai) by [Sashir Estela](https://github.com/sashirestela)
* [Spring AI](https://spring.io/projects/spring-ai)

Julia

* [OpenAI.jl](https://github.com/rory-linehan/OpenAI.jl) by [rory-linehan](https://github.com/rory-linehan)

Kotlin

* [openai-kotlin](https://github.com/Aallam/openai-kotlin) by [Mouaad Aallam](https://github.com/Aallam)

Node.js

* [openai-api](https://www.npmjs.com/package/openai-api) by [Njerschow](https://github.com/Njerschow)
* [openai-api-node](https://www.npmjs.com/package/openai-api-node) by [erlapso](https://github.com/erlapso)
* [gpt-x](https://www.npmjs.com/package/gpt-x) by [ceifa](https://github.com/ceifa)
* [gpt3](https://www.npmjs.com/package/gpt3) by [poteat](https://github.com/poteat)
* [gpts](https://www.npmjs.com/package/gpts) by [thencc](https://github.com/thencc)
* [@dalenguyen/openai](https://www.npmjs.com/package/@dalenguyen/openai) by [dalenguyen](https://github.com/dalenguyen)
* [tectalic/openai](https://github.com/tectalichq/public-openai-client-js) by [tectalic](https://tectalic.com/)

PHP

* [orhanerday/open-ai](https://packagist.org/packages/orhanerday/open-ai) by [orhanerday](https://github.com/orhanerday)

* [tectalic/openai](https://github.com/tectalichq/public-openai-client-php) by [tectalic](https://tectalic.com/)
* [openai-php client](https://github.com/openai-php/client) by [openai-php](https://github.com/openai-php)

Python

* [chronology](https://github.com/OthersideAI/chronology) by [OthersideAI](https://www.othersideai.com/)

R

* [rgpt3](https://github.com/ben-aaron188/rgpt3) by [ben-aaron188](https://github.com/ben-aaron188)

Ruby

* [openai](https://github.com/nileshtrivedi/openai/) by [nileshtrivedi](https://github.com/nileshtrivedi)
* [ruby-openai](https://github.com/alexrudall/ruby-openai) by [alexrudall](https://github.com/alexrudall)

Rust

* [async-openai](https://github.com/64bit/async-openai) by [64bit](https://github.com/64bit)
* [fieri](https://github.com/lbkolev/fieri) by [lbkolev](https://github.com/lbkolev)

Scala

* [openai-scala-client](https://github.com/cequence-io/openai-scala-client) by [cequence-io](https://github.com/cequence-io)

Swift

* [AIProxySwift](https://github.com/lzell/AIProxySwift) by [Lou Zell](https://github.com/lzell)
* [OpenAIKit](https://github.com/dylanshine/openai-kit) by [dylanshine](https://github.com/dylanshine)
* [OpenAI](https://github.com/MacPaw/OpenAI/) by [MacPaw](https://github.com/MacPaw)

Unity

* [OpenAi-Api-Unity](https://github.com/hexthedev/OpenAi-Api-Unity) by [hexthedev](https://github.com/hexthedev)
* [com.openai.unity](https://github.com/RageAgainstThePixel/com.openai.unity) by [RageAgainstThePixel](https://github.com/RageAgainstThePixel)

Unreal Engine

* [OpenAI-Api-Unreal](https://github.com/KellanM/OpenAI-Api-Unreal) by [KellanM](https://github.com/KellanM)

Other OpenAI repositories
---

* [tiktoken](https://github.com/openai/tiktoken) - counting tokens
* [simple-evals](https://github.com/openai/simple-evals) - simple evaluation library
* [mle-bench](https://github.com/openai/mle-bench) - library to evaluate machine learning engineer agents
* [gym](https://github.com/openai/gym) - reinforcement learning library
* [swarm](https://github.com/openai/swarm) - educational orchestration repository

Was this page useful? En

Docs MCP

Search and read OpenAI developer docs from your editor.

OpenAI hosts a public Model Context Protocol (MCP) server for developer documentation
on developers.openai.com and platform.openai.com.

Server URL (streamable HTTP): `https://developers.openai.com/mcp`

What it provides

* Read-only access to OpenAI developer documentation (search + page content).
* A way to pull documentation into your agent's context while you work.

This MCP server is documentation-only. It does not call the OpenAI API on your behalf.

Quickstart

Codex

You can connect Codex to [MCP servers](https://developers.openai.com/codex/mcp) in the [CLI](https://developers.openai.com/codex/cli) or [IDE extension](https://developers.openai.com/codex/ide). The configuration is shared between both so you only have to set it up once.

Add the server using the Codex CLI:

```
codex mcp add openaiDeveloperDocs --
url https://developers.openai.com/mcp
```

Verify it's configured:

```
codex mcp list
```

Alternatively, you can add it in `~/.codex/config.toml` directly:

```
[mcp_servers.openaiDeveloperDocs]
url = "https://developers.openai.com/mcp"
```

VS Code (GitHub Copilot Agent mode)

VS Code supports MCP servers when using GitHub Copilot in Agent mode.

Click the following link to add the Docs MCP to VS Code:

[Install in VS Code](https://platform.openai.com/docs/docs-mcp)

Alternatively, you can manually add a `.vscode/mcp.json` in your project root:

```
{
  "servers": {
    "openaiDeveloperDocs": {
      "type": "http",
      "url": "https://developers.openai.com/mcp"
    }
  }
}
```

To have VS Code reliably use the MCP server, add this snippet to your `AGENTS.md`:

```
Always use the OpenAI developer documentation MCP server if you need to work with the OpenAI API, ChatGPT Apps SDK, Codex, or related docs without me having to explicitly ask.
```

Open Copilot Chat, switch to **Agent** mode, enable the server in the tools picker, and ask an OpenAI-related question like:

> Look up the request schema for Responses API tools in the OpenAI developer docs and summarize the required fields.

Cursor

Cursor has native MCP support and reads configuration from `mcp.json`.

Install with Cursor:

[Install in Cursor](https://cursor.com/en-US/install-mcp?name=openaiDeveloperDocs&config=eyJ1cmwiOiAiaHR0cHM6Ly9kZXZlbG9wZXJzLm9wZW5haS5jb20vbWNwIn0%3D)

Alternatively, create a `~/.cursor/mcp.json` (macOS/Linux) and add:

```
{
  "mcpServers": {
    "openaiDeveloperDocs": {
      "url": "https://developers.openai.com/mcp"
    }
  }
}
```

To have Cursor reliably use the MCP server, add this snippet to your `AGENTS.md`:

```
Always use the OpenAI developer documentation MCP server if you need to work with the OpenAI API, ChatGPT Apps SDK, Codex, or related docs without me having to explicitly ask.
```

Restart Cursor and ask Cursor's agent an OpenAI-related question like:

> Look up the request schema for Responses API tools in the OpenAI developer docs and summarize the required fields.

Tips
---

* If you don't have the snippet in the AGENTS.md file, you need to explicitly tell your agent to consult the Docs MCP server for the answer.
* If you have more than one MCP server, keep server names short and descriptive to aid the agent in selecting the server.

Was this page useful?

Text generation
==========

Learn how to prompt a model to generate text.

With the OpenAI API, you can use a [large language model](https://platform.openai.com/docs/models) to generate text from a prompt, as you might using [ChatGPT](https://chatgpt.com). Models can generate almost any kind of text response—like code, mathematical equations, structured JSON data, or human-like prose.

Here's a simple example using the [Responses API](https://platform.openai.com/docs/api-reference/responses), our recommended API for all new projects.

Generate text from a simple prompt

```
import OpenAI from "openai";
const client = new OpenAI();

const response = await client.responses.create({
    model: "gpt-5.2",
    input: "Write a one-sentence bedtime story about a unicorn."
});

console.log(response.output_text);
```

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5.2",
    input="Write a one-sentence bedtime story about a unicorn."
)

print(response.output_text)
```

```
using System;
using System.Threading.Tasks;
using OpenAI;

class Program
{
    static async Task Main()
    {
        var client = new OpenAIClient(
            Environment.GetEnvironmentVariable("
OPENAI_API_KEY")
        );
```

```
        var response = await client.Responses.CreateAsync(new
ResponseCreateRequest
        {
            Model = "gpt-5.2",
            Input = "Say 'this is a test.'"
        });

        Console.WriteLine($"[ASSISTANT]:
{response.OutputText()}");
    }
}
```

```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;
import com.openai.models.responses.Response;
import com.openai.models.responses.ResponseCreateParams;

public class Main {
    public static void main(String[] args) {
        OpenAIClient client = OpenAIOkHttpClient.fromEnv();

        ResponseCreateParams params =
ResponseCreateParams.builder()
                .input("Say this is a test")
                .model("gpt-5.2")
                .build();

        Response response = client.responses().create(params);
        System.out.println(response.outputText());
    }
}
```

```
package main

import (
 "context"
 "fmt"

 "github.com/openai/openai-go/v3"
 "github.com/openai/openai-go/v3/option"
 "github.com/openai/openai-go/v3/responses"
)

func main() {
 client := openai.NewClient(
  option.WithAPIKey("My API Key"), // or set OPENAI_API_KEY in
your env
 )

 resp, err := client.Responses.New(context.TODO(),
openai.ResponseNewParams{
  Model: "gpt-5.2",
  Input: responses.ResponseNewParamsInputUnion{OfString:
openai.String("Say this is a test")},
 })
 if err != nil {
  panic(err.Error())
 }

 fmt.Println(resp.OutputText())
}
```

```
curl "https://api.openai.com/v1/responses" \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer $OPENAI_API_KEY" \
```

```
  -d '{
    "model": "gpt-5.2",
    "input": "Write a one-sentence bedtime story about a
unicorn."
  }'
```

An array of content generated by the model is in the `output` property of the response. In this simple example, we have just one output which looks like this:

```
[
  {
    "id": "msg_67b73f697ba4819183a15cc17d011509",
    "type": "message",
    "role": "assistant",
    "content": [
      {
        "type": "output_text",
        "text": "Under the soft glow of the moon, Luna the
unicorn danced through fields of twinkling stardust, leaving
trails of dreams for every child asleep.",
        "annotations": []
      }
    ]
  }
]
```

The `output` array often has more than one item in it! It can contain tool calls, data about reasoning tokens generated by [reasoning models](https://platform.openai.com/docs/guides/reasoning), and other items. It is not safe to assume that the model's text output is present at `output[0].content[0].text`.

Some of our [official SDKs](https://platform.openai.com/docs/libraries) include an `output_text` property on model responses for convenience, which aggregates all text outputs from the model into a single string. This may be useful as a shortcut to access text output from the model.

In addition to plain text, you can also have the model return structured data in JSON format—this feature is called [**Structured Outputs**](https://platform.openai.com/docs/guides/structured-outputs).

Prompt engineering

Prompt engineering is the process of writing effective instructions for a model, such that it consistently generates content that meets your requirements.

Because the content generated from a model is non-deterministic, prompting to get your desired output is a mix of art and science. However, you can apply techniques and best practices to get good results consistently.

Some prompt engineering techniques work with every model, like using message roles. But different models might need to be prompted differently to produce the best results. Even different snapshots of models within the same family could produce different results. So as you build more complex applications, we strongly recommend:

* Pinning your production applications to specific [model snapshots](https://platform.openai.com/docs/models) (like `gpt-5-2025-08-07` for example) to ensure consistent behavior
* Building [evals](https://platform.openai.com/docs/guides/evals) that measure the behavior of

your prompts so you can monitor prompt performance as you iterate, or when you change and upgrade model versions

Now, let's examine some tools and techniques available to you to construct prompts.

Choosing models and APIs

OpenAI has many different [models](https://platform.openai.com/docs/models) and several APIs to choose from. [Reasoning models](https://platform.openai.com/docs/guides/reasoning), like o3 and GPT-5, behave differently from chat models and respond better to different prompts. One important note is that reasoning models perform better and demonstrate higher intelligence when used with the Responses API.

If you're building any text generation app, we recommend using the Responses API over the older Chat Completions API. And if you're using a reasoning model, it's especially useful to [migrate to Responses](https://platform.openai.com/docs/guides/migrate-to-responses).

Message roles and instruction following

You can provide instructions to the model with [differing levels of authority](https://model-spec.openai.com/2025-02-12.html#chain_of_command) using the `instructions` API parameter along with **message roles**.

The `instructions` parameter gives the model high-level instructions on how it should behave while generating a response, including tone, goals, and examples of correct responses. Any instructions provided this way will take priority over a prompt in the `input` parameter.

Generate text with instructions

```
import OpenAI from "openai";
const client = new OpenAI();

const response = await client.responses.create({
    model: "gpt-5",
    reasoning: { effort: "low" },
    instructions: "Talk like a pirate.",
    input: "Are semicolons optional in JavaScript?",
});

console.log(response.output_text);
```

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5",
    reasoning={"effort": "low"},
    instructions="Talk like a pirate.",
    input="Are semicolons optional in JavaScript?",
)

print(response.output_text)
```

```
curl "https://api.openai.com/v1/responses" \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer $OPENAI_API_KEY" \
    -d '{
```

```
        "model": "gpt-5",
        "reasoning": {"effort": "low"},
        "instructions": "Talk like a pirate.",
        "input": "Are semicolons optional in JavaScript?"
    }'
```

The example above is roughly equivalent to using the following input messages in the `input` array:

Generate text with messages using different roles

```
import OpenAI from "openai";
const client = new OpenAI();

const response = await client.responses.create({
    model: "gpt-5",
    reasoning: { effort: "low" },
    input: [
        {
            role: "developer",
            content: "Talk like a pirate."
        },
        {
            role: "user",
            content: "Are semicolons optional in JavaScript?",
        },
    ],
});

console.log(response.output_text);
```

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5",
    reasoning={"effort": "low"},
    input=[
        {
            "role": "developer",
            "content": "Talk like a pirate."
        },
        {
            "role": "user",
            "content": "Are semicolons optional in JavaScript?"
        }
    ]
)

print(response.output_text)
```

```
curl "https://api.openai.com/v1/responses" \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer $OPENAI_API_KEY" \
    -d '{
        "model": "gpt-5",
        "reasoning": {"effort": "low"},
        "input": [
            {
                "role": "developer",
                "content": "Talk like a pirate."
            },
            {
                "role": "user",
                "content": "Are semicolons optional in JavaScript?"
            }
```

```
    ]
  }'
```

Note that the `instructions` parameter only applies to the current response generation request. If you are [managing conversation state](https://platform.openai.com/docs/guides/conversation-state) with the `previous_response_id` parameter, the `instructions` used on previous turns will not be present in the context.

The [OpenAI model spec](https://model-spec.openai.com/2025-02-12.html#chain_of_command) describes how our models give different levels of priority to messages with different roles.

developer	user	assistant
developer messages are instructions provided by the application developer, prioritized ahead of user messages.	user messages are instructions provided by an end user, prioritized behind developer messages.	Messages generated by the model have the assistant role.

A multi-turn conversation may consist of several messages of these types, along with other content types provided by both you and the model. Learn more about [managing conversation state here](https://platform.openai.com/docs/guides/conversation-state).

You could think about `developer` and `user` messages like a function and its arguments in a programming language.

* `developer` messages provide the system's rules and business logic, like a function definition.
* `user` messages provide inputs and configuration to which the `developer` message instructions are applied, like arguments to a function.

Reusable prompts

In the OpenAI dashboard, you can develop reusable [prompts](https://platform.openai.com/chat/edit) that you can use in API requests, rather than specifying the content of prompts in code. This way, you can more easily build and evaluate your prompts, and deploy improved versions of your prompts without changing your integration code.

Here's how it works:

1. **Create a reusable prompt** in the [dashboard](https://platform.openai.com/chat/edit) with placeholders like `{{customer_name}}`.
2. **Use the prompt** in your API request with the `prompt` parameter. The prompt parameter object has three properties you can configure:
   * `id` — Unique identifier of your prompt, found in the dashboard
   * `version` — A specific version of your prompt (defaults to the "current" version as specified in the dashboard)
   * `variables` — A map of values to substitute in for variables in your prompt. The substitution values can either be strings, or other Response input message types like `input_image` or `input_file`. [See the full API reference](https://platform.openai.com/docs/api-reference/responses/create).

String variables

Generate text with a prompt template

```
import OpenAI from "openai";
```

```
const client = new OpenAI();

const response = await client.responses.create({
    model: "gpt-5",
    prompt: {
        id: "pmpt_abc123",
        version: "2",
        variables: {
            customer_name: "Jane Doe",
            product: "40oz juice box"
        }
    }
});

console.log(response.output_text);
```

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5",
    prompt={
        "id": "pmpt_abc123",
        "version": "2",
        "variables": {
            "customer_name": "Jane Doe",
            "product": "40oz juice box"
        }
    }
)

print(response.output_text)
```

```
curl https://api.openai.com/v1/responses \
  -H "Authorization: Bearer $OPENAI_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gpt-5",
    "prompt": {
      "id": "pmpt_abc123",
      "version": "2",
      "variables": {
        "customer_name": "Jane Doe",
        "product": "40oz juice box"
      }
    }
  }'
```

Variables with file input

Prompt template with file input variable

```
import fs from "fs";
import OpenAI from "openai";
const client = new OpenAI();

// Upload a PDF we will reference in the prompt variables
const file = await client.files.create({
    file: fs.createReadStream("draconomicon.pdf"),
    purpose: "user_data",
});

const response = await client.responses.create({
    model: "gpt-5",
    prompt: {
        id: "pmpt_abc123",
```

```
        variables: {
            topic: "Dragons",
            reference_pdf: {
                type: "input_file",
                file_id: file.id,
            },
        },
    },
});

console.log(response.output_text);
```

```
import openai, pathlib

client = openai.OpenAI()

# Upload a PDF we will reference in the variables
file = client.files.create(
    file=open("draconomicon.pdf", "rb"),
    purpose="user_data",
)

response = client.responses.create(
    model="gpt-5",
    prompt={
        "id": "pmpt_abc123",
        "variables": {
            "topic": "Dragons",
            "reference_pdf": {
                "type": "input_file",
                "file_id": file.id,
            },
        },
    },
)

print(response.output_text)
```

```
# Assume you have already uploaded the PDF and obtained
FILE_ID
curl https://api.openai.com/v1/responses -H "Authorization:
Bearer $OPENAI_API_KEY" -H "Content-Type: application/json" -
d '{
    "model": "gpt-5",
    "prompt": {
      "id": "pmpt_abc123",
      "variables": {
        "topic": "Dragons",
        "reference_pdf": {
          "type": "input_file",
          "file_id": "file-abc123"
        }
      }
    }
  }'
```

Next steps

Now that you known the basics of text inputs and outputs, you might want to check out one of these resources next.

[

Build a prompt in the Playground

Use the Playground to develop and iterate on prompts.

](https://platform.openai.com/chat/edit)[

Generate JSON data with Structured Outputs

Ensure JSON data emitted from a model conforms to a JSON schema.

](https://platform.openai.com/docs/guides/structured-outputs)[

Full API reference

Check out all the options for text generation in the API reference.

](https://platform.openai.com/docs/api-reference/responses)

Was this page useful?

Code generation
==========

Learn how to use OpenAI Codex models to generate code.

Writing, reviewing, editing, and answering questions about code is one of the primary use cases for OpenAI models today. This guide walks through your options for code generation.

Codex is OpenAI's series of AI coding tools that help developers move faster by delegating tasks to powerful cloud and local coding agents. Interact with Codex in a variety of interfaces: in your IDE, through the CLI, on web and mobile sites, or in your CI/CD pipelines with the SDK. Codex is the best way to get agentic software engineering on your projects.

Codex models are LLMs specifically trained at coding tasks. They power Codex, and you can use them to create coding-specific applications. For example, let your _end users_ generate code.

Get started

[

![Use Codex for out-of-the-box coding agents](https://cdn.openai.com/API/docs/images/build.png)

Use Codex for out-of-the-box coding agents

Connect your codebase Codex and accelerate your projects using software engineering agents.

](https://platform.openai.com/docs/guides/code-generation#use-codex)[

![Integrate with coding models](https://cdn.openai.com/API/docs/images/build.png)

Integrate with coding models

Use OpenAI models in your application. Add them to a model picker, for instance.

](https://platform.openai.com/docs/guides/code-generation#integrate-with-coding-models)

Use Codex

Codex has an interface in the browser, similar to ChatGPT, where you can kick off coding tasks that run in the cloud. Visit chatgpt.com/codex to use it.

Codex also has an IDE extension, CLI, and SDK to help you create coding tasks in whichever environment makes the most sense for you. For example, the SDK is useful for using Codex in CI/CD pipelines. The CLI, on the other hand, runs locally from your terminal and can read, modify, and run code on your machine.

See the [Codex docs](https://developers.openai.com/codex) for quickstarts, reference, pricing, and more information.

Integrate with coding models

OpenAI has several models trained specifically to work with code. GPT-5.1-Codex-Max is our best agentic coding model. That said, many OpenAI models excel at writing and editing code as well as other tasks. Use a Codex model if you _only_ want it for coding-related work.

Here's an example that calls GPT-5.1-Codex-Max, the model that powers Codex:

Slower, high reasoning tasks

```
import OpenAI from "openai";
const openai = new OpenAI();

const result = await openai.responses.create({
  model: "gpt-5.1-codex-max",
  input: "Find the null pointer exception: ...your code here...",
  reasoning: { effort: "high" },
});

console.log(result.output_text);
```

```
from openai import OpenAI
client = OpenAI()

result = client.responses.create(
    model="gpt-5.1-codex-max",
    input="Find the null pointer exception: ...your code here...",
    reasoning={ "effort": "high" },
)

print(result.output_text)
```

```
curl https://api.openai.com/v1/responses \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $OPENAI_API_KEY" \
  -d '{
    "model": "gpt-5.1-codex-max",
    "input": "Find the null pointer exception: ...your code here...",
    "reasoning": { "effort": "high" }
  }'
```

Learn more about GPT-5.1-Codex-Max in the [blog post](https://openai.com/index/gpt-5-1-codex-max/). Read the [GPT-5.1-Codex-Max prompting guide](https://cookbook.openai.com/examples/gpt-5/gpt-5-1-codex-max_prompting_guide) to start building with it.

Next steps
---

* Visit the [Codex docs](https://developers.openai.com/codex) to learn what you can do with Codex, set up Codex in whichever interface you choose, or find more details.
* See the [GPT-5.1.-Codex-Max prompting guide](https://cookbook.openai.com/examples/gpt-5/gpt-5-1-codex-max_prompting_guide) for best practices on maximizing performance.
* Learn more about [GPT-5.1 and its variants](https://platform.openai.com/docs/guides/latest-model).
* Visit the [GPT-5.1-Codex-Max model page](https://platform.openai.com/docs/models/gpt-5.1-codex-max) to compare models.

Was this page useful?

Audio and speech
==========

Explore audio and speech features in the OpenAI API.

The OpenAI API provides a range of audio capabilities. If you know what you want to build, find your use case below to get started. If you're not sure where to start, read this page as an overview.

Build with audio

[

![Build voice agents](https://cdn.openai.com/API/docs/images/voice-agents-rounded.png)

Build voice agents

Build interactive voice-driven applications.

](https://platform.openai.com/docs/guides/voice-agents)[

![Transcribe audio](https://cdn.openai.com/API/docs/images/stt-rounded.png)

Transcribe audio

Convert speech to text instantly and accurately.

](https://platform.openai.com/docs/guides/speech-to-text)[

![Speak text](https://cdn.openai.com/API/docs/images/tts-rounded.png)

Speak text

Turn text into natural-sounding speech in real time.

](https://platform.openai.com/docs/guides/text-to-speech)

A tour of audio use cases

LLMs can process audio by using sound as input, creating sound as output, or both. OpenAI has several API endpoints that help you build audio applications or voice agents.

Voice agents

Voice agents understand audio to handle tasks and respond back in natural language. There are two main ways to approach voice agents: either with speech-to-speech models and the [Realtime API](https://platform.openai.com/docs/guides/realtime), or by chaining together a speech-to-text model, a text language model to process the request, and a text-to-speech model to respond. Speech-to-speech is lower latency and more natural, but chaining together a voice agent is a reliable way to extend a text-based agent into a voice agent. If you are already using the [Agents SDK](https://platform.openai.com/docs/guides/agents), you can [extend your existing agents with voice capabilities](https://openai.github.io/openai-agents-python/voice/quickstart/) using the chained approach.

Streaming audio

Process audio in real time to build voice agents and other low-latency applications, including transcription use cases. You can stream audio in and out of a model with the [Realtime API](https://platform.openai.com/docs/guides/realtime). Our advanced speech models provide automatic speech recognition for improved accuracy, low-latency interactions, and multilingual support.

Text to speech

For turning text into speech, use the [Audio API](https://platform.openai.com/docs/api-reference/audio/) `audio/speech` endpoint. Models compatible with this endpoint are `gpt-4o-mini-tts`, `tts-1`, and `tts-1-hd`. With `gpt-4o-mini-tts`, you can ask the model to speak a certain way or with a certain tone of voice.

Speech to text

For speech to text, use the [Audio API](https://platform.openai.com/docs/api-reference/audio/) `audio/transcriptions` endpoint. Models compatible with this endpoint are `gpt-4o-transcribe`, `gpt-4o-mini-transcribe`, `whisper-1`, and `gpt-4o-transcribe-diarize`. `gpt-4o-transcribe-diarize` adds speaker labels and timestamps for HTTP requests and is intended for non-latency-sensitive workloads, while the other models focus on transcription only. With streaming, you can continuously pass in audio and get a continuous stream of text back.

Choosing the right API

There are multiple APIs for transcribing or generating audio:

API	Supported modalities	Streaming support
Realtime API	Audio and text inputs and outputs	Audio streaming in, audio and text streaming out
Chat Completions API	Audio and text inputs and outputs	Audio and text streaming out
Transcription API	Audio inputs	Text streaming out
Speech API	Text inputs and audio outputs	Audio streaming out

General use APIs vs. specialized APIs

The main distinction is general use APIs vs. specialized APIs. With the Realtime and Chat Completions APIs, you can use our latest models' native audio understanding and generation capabilities and combine them with other features like function calling. These APIs can be used for a wide range of use cases, and you can select the model you want to use.

On the other hand, the Transcription, Translation and Speech APIs are specialized to work with specific models and only meant for one purpose.

Talking with a model vs. controlling the script

Another way to select the right API is asking yourself how much control you need. To design conversational interactions, where the model thinks and responds in speech, use the Realtime or Chat Completions API, depending if you need low-latency or not.

You won't know exactly what the model will say ahead of time, as it will generate audio responses directly, but the conversation will feel natural.

For more control and predictability, you can use the Speech-to-text / LLM / Text-to-speech pattern, so you know exactly what the model will say and can control the response. Please note that with this method, there will be added latency.

This is what the Audio APIs are for: pair an LLM with the `audio/transcriptions` and `audio/speech` endpoints to take spoken user input, process and generate a text response, and then convert that to speech that the user can hear.

Recommendations

* If you need [real-time interactions](https://platform.openai.com/docs/guides/realtime-conversations) or [transcription](https://platform.openai.com/docs/guides/realtime-transcription), use the Realtime API.
* If realtime is not a requirement but you're looking to build a [voice agent](https://platform.openai.com/docs/guides/voice-agents) or an audio-based application that requires features such as [function calling](https://platform.openai.com/docs/guides/function-calling), use the Chat Completions API.
* For use cases with one specific purpose, use the Transcription, Translation, or Speech APIs.

Add audio to your existing application

Models such as `gpt-realtime` and `gpt-audio` are natively multimodal, meaning they can understand and generate multiple modalities as input and output.

If you already have a text-based LLM application with the [Chat Completions endpoint](https://platform.openai.com/docs/api-reference/chat/), you may want to add audio capabilities. For example, if your chat application supports text input, you can add audio input and output—just include `audio` in the `modalities` array and use an audio model, like `gpt-audio`.

Audio is not yet supported in the [Responses API](https://platform.openai.com/docs/api-reference/chat/completions/responses).

Audio output from model

Create a human-like audio response to a prompt

```
import { writeFileSync } from "node:fs";
```

```
import OpenAI from "openai";

const openai = new OpenAI();

// Generate an audio response to the given prompt
const response = await openai.chat.completions.create({
  model: "gpt-audio",
  modalities: ["text", "audio"],
  audio: { voice: "alloy", format: "wav" },
  messages: [
    {
      role: "user",
      content: "Is a golden retriever a good family dog?"
    }
  ],
  store: true,
});

// Inspect returned data
console.log(response.choices[0]);

// Write audio data to a file
writeFileSync(
  "dog.wav",
  Buffer.from(response.choices[0].message.audio.data,
'base64'),
  { encoding: "utf-8" }
);
```

```
import base64
from openai import OpenAI

client = OpenAI()

completion = client.chat.completions.create(
    model="gpt-audio",
    modalities=["text", "audio"],
    audio={"voice": "alloy", "format": "wav"},
    messages=[
        {
            "role": "user",
            "content": "Is a golden retriever a good family dog?"
        }
    ]
)

print(completion.choices[0])

wav_bytes = base64.b64decode(completion.
choices[0].message.audio.data)
with open("dog.wav", "wb") as f:
    f.write(wav_bytes)
```

```
curl "https://api.openai.com/v1/chat/completions" \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer $OPENAI_API_KEY" \
    -d '{
      "model": "gpt-audio",
      "modalities": ["text", "audio"],
      "audio": { "voice": "alloy", "format": "wav" },
      "messages": [
        {
          "role": "user",
          "content": "Is a golden retriever a good family dog?"
        }
      ]
    }'
```

Audio input to model

Use audio inputs for prompting a model

```
import OpenAI from "openai";
const openai = new OpenAI();

// Fetch an audio file and convert it to a base64 string
const url = "https://cdn.openai.com/API/docs/audio/alloy.wav";
const audioResponse = await fetch(url);
const buffer = await audioResponse.arrayBuffer();
const base64str = Buffer.from(buffer).toString("base64");

const response = await openai.chat.completions.create({
  model: "gpt-audio",
  modalities: ["text", "audio"],
  audio: { voice: "alloy", format: "wav" },
  messages: [
    {
      role: "user",
      content: [
        { type: "text", text: "What is in this recording?" },
        { type: "input_audio", input_audio: { data: base64str, format:
"wav" }}
      ]
    }
  ],
  store: true,
});

console.log(response.choices[0]);
```

```
import base64
import requests
from openai import OpenAI

client = OpenAI()

# Fetch the audio file and convert it to a base64 encoded string
url = "https://cdn.openai.com/API/docs/audio/alloy.wav"
response = requests.get(url)
response.raise_for_status()
wav_data = response.content
encoded_string = base64.b64encode(wav_data).decode('utf-8')

completion = client.chat.completions.create(
    model="gpt-audio",
    modalities=["text", "audio"],
    audio={"voice": "alloy", "format": "wav"},
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "text",
                    "text": "What is in this recording?"
                },
                {
                    "type": "input_audio",
                    "input_audio": {
                        "data": encoded_string,
                        "format": "wav"
                    }
                }
            ]
        },
    ]
)
```

```
print(completion.choices[0].message)
```

```
curl "https://api.openai.com/v1/chat/completions" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $OPENAI_API_KEY" \
  -d '{
    "model": "gpt-audio",
    "modalities": ["text", "audio"],
    "audio": { "voice": "alloy", "format": "wav" },
    "messages": [
      {
        "role": "user",
        "content": [
          { "type": "text", "text": "What is in this recording?" },
          {
            "type": "input_audio",
            "input_audio": {
              "data": "<base64 bytes here>",
              "format": "wav"
            }
          }
        ]
      }
    ]
  }'
```

Was this page useful?

https://json-schema.org/overview/what-is-jsonschema

https://github.com/openai/openai-python/blob/main/helpers.md#structured-outputs-parsing-helpers https://github.com/openai/openai-node/blob/master/helpers.md#structured-outputs-parsing-helpers https://docs.pydantic.dev/latest/ https://zod.dev/

https://platform.openai.com/docs/models

https://platform.openai.com/docs/guides/structured-outputs#json-mode

https://platform.openai.com/docs/guides/function-calling

https://platform.openai.com/docs/guides/function-calling#function-calling-with-structured-outputs

https://platform.openai.com/docs/guides/structured-outputs#json-mode

https://api.openai.com/v1/responses

https://api.openai.com/v1/responses

https://api.openai.com/v1/responses

https://api.openai.com/v1/responses